Przegląd literatury — Routing LLM

Notatki do pracy inżynierskiej

Artykuł 1: RouteLLM

Ong et al., 2024 — "RouteLLM: Learning to Route LLMs with Preference Data"

---

1. Problem i motywacja

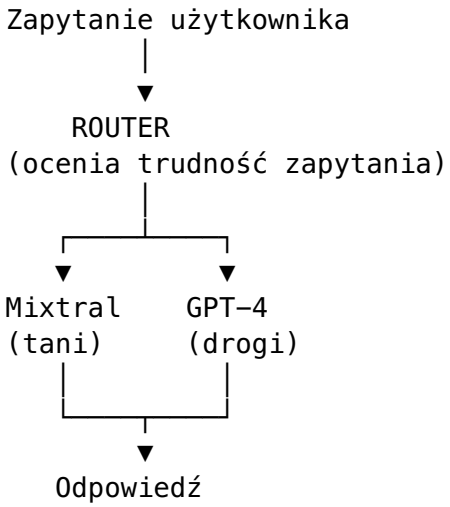
Duże modele językowe dzielą się na dwie kategorie: - **Mocne** (np. GPT-4) — wysokiej jakości odpowiedzi, ale bardzo drogie - **Słabe** (np. Mixtral 8x7B) — tańsze, ale gorszej jakości

Firmy stają przed wyborem: albo zawsze używają drogiego modelu (przepłacają), albo taniego (tracą jakość). Autorzy proponują **router** — system który dla każdego zapytania automatycznie decyduje do którego modelu je wysłać.

Kluczowa obserwacja: **nie każde zapytanie wymaga mocnego modelu**. Proste pytania słaby model obsługuje równie dobrze. Tylko trudne zapytania faktycznie wymagają GPT-4.

---

2. Jak działa routing



Router dla każdego zapytania q oblicza prawdopodobieństwo:

```
P(win_strong | q) = szansa że mocny model da lepszą odpowiedź
```

Jeśli to prawdopodobieństwo przekracza próg α → zapytanie idzie do mocnego modelu. Próg α jest konfigurowalny: wyższy = więcej do słabego (taniej), niższy = więcej do mocnego.

---

3. Dane treningowe

Router musi się nauczyć które pytania są trudne. Autorzy korzystają z trzech źródeł:

Źródło	Rozmiar	Skąd etykiety	Koszt
Chatbot Arena	80K	ludzie głosowali	darmowe
MMLU	~1500	gotowy klucz odpowiedzi	darmowe
Nectar	~120K	GPT-4 jako sędzia	~700 USD

Chatbot Arena — strona gdzie użytkownicy zadają pytania dwóm anonimowym modelom i głosują który odpowiedział lepiej. 80 000 głosowań zebranych przez twórców Areny. Wszystkie trzy zbiory są publicznie dostępne na Hugging Face.

Problem z samymi danymi Areny: pytania na Arenie są głównie konwersacyjne i ogólne. Gdy testują router na matematyce lub wiedzy ogólnej — wyniki bez augmentacji są słabe, bo rozkład danych nie pasuje.

Rozwiązanie — augmentacja: - MMLU: gotowe pytania z kluczem odpowiedzi, zero kosztów - Nectar: pytania otwarte, etykiety wygenerowane przez GPT-4 Judge (~700 USD)

---

4. Cztery metody routera

Metoda 1 — SW Ranking (Similarity-Weighted)

Nie wymaga żadnego trenowania. Działa przez analogię:

```
Nowe zapytanie → zamień na wektor (embedding)
                          │
                          ▼
        porównaj z 80K pytań z Areny (cosine similarity)
                          │
                          ▼
        podobne pytania mają większą wagę
                          │
                          ▼
        policz ważoną sumę głosów: ile razy mocny wygrał?
                          │
```

```
                    ▼
          P(win_strong | q)
```

Cechy: zero kosztów trenowania, ale wolny w działaniu i drogi operacyjnie (~39 USD/milion zapytań) bo przy każdym zapytaniu przeszukuje całą historię.

Metoda 2 — Matrix Factorization

Inspirowane systemami rekomendacyjnymi (jak Netflix). Uczy się ukrytych reprezentacji modeli i zapytań.

```
Trenowanie:
  dane: (zapytanie, który model wygrał)
  uczy się wektora dla każdego modelu i zapytania
  jakość odpowiedzi = iloczyn wektorów modelu i zapytania

Inferencja:
  nowe zapytanie → embedding → wyuczona funkcja jakości
                                          │
                                          ▼
                                P(win_strong | q)
```

Cechy: wymaga trenowania (GPU 8GB, ~10 epok), bardzo szybki w inferencji (~155 req/s), tani operacyjnie (~3.32 USD/milion).

Metoda 3 — BERT Classifier

Klasyczny model językowy (BERT) fine-tunowany do klasyfikacji trudności.

```
Trenowanie:
  fine-tuning BERT_base na danych preferencji
  ostatnia warstwa: P(win_strong | q)

Inferencja:
  zapytanie → tokenizacja → BERT encoder
                                  │
                              token [CLS]
                                  │
                            warstwa liniowa
                                  │
                          P(win_strong | q)
```

Cechy: fine-tuning (~2000 kroków, 2x GPU L4), szybki (~70 req/s), tani (~3.19 USD/milion). Bez augmentacji działa gorzej niż losowy wybór.

Metoda 4 — Causal LLM Classifier

Llama 3 8B fine-tunowana jako klasyfikator. Model odpowiada jednym tokenem: “strong” lub “weak”.

```
Trenowanie:
  fine-tuning Llama 3 8B
  format: [System: jesteś routerem] [User: zapytanie] [Assistant:
"strong"/"weak"]

Inferencja:
  zapytanie → Llama 3 8B → softmax na tokenach "strong"/"weak"
                                          |
                                  P(win_strong | q)
```

Cechy: fine-tuning (~2000 kroków, 8x GPU A100), najwolniejszy (~42 req/s), najlepiej rozumie kontekst.

Porównanie metod

Metoda	Trenowanie	Szybkość	Koszt/M
SW Ranking	brak	wolna	$39.26
Matrix Factor.	tak (mały)	~155 req/s	$3.32
BERT	fine-tuning	~70 req/s	$3.19
Causal LLM	fine-tuning	~42 req/s	$5.23

Ważne: fine-tuning dotyczy tylko **routera** (małego klasyfikatora), nie modeli które odpowiadają użytkownikom (GPT-4, Mixtral używane są as-is).

---

5. Metryki ewaluacji

Cost (c) — jaki procent zapytań trafił do mocnego modelu. Im niższy tym taniej.

PGR (Performance Gap Recovered) — ile procent różnicy jakości między słabym a mocnym modelem router odzyskuje:

```
PGR = (jakość_routera − jakość_słabego) / (jakość_mocnego −
jakość_słabego)
```

Przykład liczbowy:

```
jakość_słabego  = 6.5/10
jakość_mocnego  = 9.0/10
jakość_routera  = 8.2/10

PGR = (8.2 − 6.5) / (9.0 − 6.5) = 68%
```

APGR — średnie PGR liczone przez cały zakres kosztów (od 0% do 100%). Jeden numer podsumowujący jakość routera niezależnie od wybranego progu.

CPT(X%) — ile minimum procent zapytań musi trafić do mocnego modelu żeby osiągnąć PGR = X%. Im niższy CPT tym lepszy router — osiąga dobrą jakość mniejszym kosztem.

---

6. Wyniki

Bez augmentacji (tylko dane Areny)

Na MT Bench (konwersacyjny) — jako tako, bo dane Areny są podobne. Na MMLU i GSM8K — prawie losowe wyniki. Router nie działa bo rozkład danych nie pasuje.

Z augmentacją — najlepsze wyniki

Benchmark	Router	CPT(50%)	APGR
MT Bench	Matrix Factorization	13.4%	0.802
MMLU	SW Ranking	35.4%	~0.70
GSM8K	Causal LLM	33.6%	~0.70

Przekładając na oszczędności (GPT-4 vs Mixtral)

Benchmark	Oszczędność	Zachowana jakość GPT-4
MT Bench	3.66x taniej	95%
MMLU	1.41x taniej	92%
GSM8K	1.49x taniej	87%

Transfer learning

Router trenowany na GPT-4 vs Mixtral przetestowany **bez ponownego trenowania** na innych parach modeli:

Para modeli	CPT(50%)	APGR
Claude Opus vs Sonnet	31.86%	0.762
Llama 3.1 70B vs 8B	25.83%	0.738

Wyniki porównywalne z oryginalną parą — router nauczył się rozpoznawać trudność zapytania, a nie specyfikę konkretnych modeli.

---

7. Uwagi krytyczne

Benchmarki są zbyt łatwe MT Bench (80 pytań konwersacyjnych), MMLU (wiedza podręcznikowa), GSM8K (matematyka szkolna) — to nie są trudne zadania dla GPT-4. Jeśli słaby model i tak dobrze radzi sobie z większością pytań, router nie musi być mądry żeby osiągać dobre wyniki. Autorzy sami przyznają to jako ograniczenie.

Rozkład produkcyjny może się różnić Wyniki na benchmarkach nie gwarantują podobnych wyników na rzeczywistych zapytaniach użytkowników z konkretnej domeny (medycyna, prawo, kodowanie specjalistyczne).

Tylko routing binarny Dwa modele — mocny i słaby. Nie ma możliwości routingu do wielu modeli jednocześnie ani Map-Reduce dla zapytań wieloetapowych.

Niewyjaśniona wariancja Ten sam router trenowany kilka razy na tych samych danych daje różne wyniki bez jasnej przyczyny.

---

8. Szacunkowe koszty wdrożenia

Autorzy podają sprzęt użyty do trenowania, ale nie przeliczają go na dolary. Poniższe oszacowania oparte na cenach chmurowych (AWS/GCP).

```
Dane treningowe:
  Chatbot Arena       $0      (publiczne)
  MMLU                $0      (publiczne)
  Nectar labels       ~$700   (wywołania GPT-4 Judge, podane w
artykule)

Fine-tuning routerów:
  SW Ranking          $0      (brak trenowania)
  Matrix Factor.      ~$5     (1x GPU 8GB, kilkadziesiąt minut)
  BERT                ~$5     (2x GPU L4, ~2000 kroków)
  Causal LLM          ~$50    (8x GPU A100, ~2000 kroków, 1–2h)
```

Koszt całkowity (dane + trening):

```
SW Ranking          $0
Matrix Factor.      ~$705
BERT                ~$705
Causal LLM          ~$750
```

Dane treningowe (Nectar labels ~$700) dominują nad kosztem samego fine-tuningu. Jest to koszt jednorazowy — dla porównania GPT-4 przy 10M tokenów dziennie to ~$300/dzień, więc router zwraca się w ciągu kilku dni użytkowania.

---

9. Znaczenie dla pracy inżynierskiej

Metryki do zastosowania: PGR, APGR i CPT są dobrze zdefiniowane i można je bezpośrednio zastosować w benchmarkach własnego systemu. Pozwala to na porównanie wyników z RouteLLM w pracy.

Dane do pobrania: Wszystkie trzy zbiory dostępne publicznie na Hugging Face: - `lmsys/chatbot_arena_conversations` - `cais/mmlu` - `berkeley-nest/Nectar`

Przewaga własnego podejścia: Własny system (kaskada heurystyk + embeddings + zero-shot) nie wymaga fine-tuningu routera ani danych preferencji. Dodatkowo obsługuje zapytania wieloetapowe przez Map-Reduce — czego RouteLLM w ogóle nie adresuje. Świadomy dobór trudniejszych danych testowych może dać bardziej wiarygodne wyniki niż benchmarki użyte w RouteLLM.

Artykuł 2: FrugalGPT

Chen, Zaharia, Zou — 2023 — "FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance"

---

1. Problem i motywacja

Podobny do RouteLLM, ale szerzej postawiony — nie tylko "który model wybrać", ale generalnie "jak używać LLM API oszczędnie".

```
GPT-4:   10M tokenów = $30
GPT-J:   10M tokenów = $0.20
                       150x różnicy ceny
```

ChatGPT kosztuje operatora >$700 000 dziennie. Małe firmy płacą >$21 000 miesięcznie za GPT-4 w obsłudze klienta. Autorzy formułują problem jako: osiągnij maksymalną jakość przy zadanym budżecie `b`.

---

2. Trzy strategie oszczędności

Główny wkład teoretyczny — taksonomia podejść do redukcji kosztów LLM.

Strategia 1 — Prompt Adaptation

Skróć prompt, zapłać mniej (koszt liniowy z długością).

Selekcja przykładów — zamiast dawać modelowi 10 przykładów few-shot, zostaw tylko niezbędne minimum.

Query Concatenation — zamiast wysyłać podobne zapytania osobno, zbij je w jedno wywołanie:

```
Zamiast 3 osobnych wywołań:
  wywołanie 1: "przetłumacz X"
  wywołanie 2: "przetłumacz Y"
  wywołanie 3: "przetłumacz Z"

Jedno wywołanie:
  "przetłumacz: X, Y, Z"
```

Strategia 2 — LLM Approximation

Zastąp drogi model czymś tańszym.

Cache odpowiedzi — zapisujesz odpowiedzi i przy podobnym pytaniu zwracasz z cache zamiast wołać API. Szczególnie przydatne gdy wielu użytkowników pyta o to samo.

Fine-tuning małego modelu — zbierasz odpowiedzi drogiego modelu na próbce, douczasz na nich mały tani model, potem używasz tylko małego. Drogi model potrzebny tylko raz.

Strategia 3 — LLM Cascade (implementowana w FrugalGPT)

Zamiast wybierać jeden model — próbujesz modeli od najtańszego, zatrzymujesz się gdy wynik jest wystarczająco dobry.

```
Zapytanie
    │
    ▼
GPT-J (najtańszy)
    │
    ▼
Scoring Function: czy odpowiedź jest wystarczająco dobra?
    │
  score > próg? → TAK → zwróć odpowiedź
                → NIE ↓
             J1-L (droższy)
                   │
             Scoring Function
                   │
             score > próg? → TAK → zwróć odpowiedź
                           → NIE ↓
                        GPT-4 (najdroższy)
                              │
                        zwróć odpowiedź
```

---

3. Kluczowy element: Scoring Function

Serce całego systemu. Mały model (DistilBERT) który dla pary (zapytanie, odpowiedź) zwraca liczbę 0–1 oznaczającą "jak bardzo ufamy tej odpowiedzi".

```
(zapytanie, odpowiedź_z_GPT-J) → DistilBERT → 0.94
                                                 |
                                      > próg 0.96? NIE → idź dalej
do droższego modelu
```

DistilBERT trenowany jest jako model regresji przewidujący poprawność odpowiedzi. Wymaga labeled data z docelowego rozkładu zapytań — jednorazowy koszt.

Kluczowa różnica względem RouteLLM: Scoring Function ocenia **odpowiedź** (po wywołaniu modelu), a nie samo zapytanie.

---

4. Dane i eksperymenty

Zbiór	Zadanie	Rozmiar
HEADLINES	klasyfikacja wiadomości fin.	10 000
OVERRULING	dokumenty prawne	2 400
COQA	reading comprehension	7 982

Testują 12 modeli od 5 dostawców: GPT-4, GPT-3, ChatGPT, GPT-Curie (OpenAI), J1-Large/Grande/Jumbo (AI21), Xlarge (Cohere), GPT-J, GPT-NeoX i inne.

---

5. Wyniki

Zbiór	Oszczędność	Jakość względem GPT-4
HEADLINES	98.2%	zachowana (lekko lepsza)
OVERRULING	73.3%	zachowana
COQA	59.2%	zachowana

Konkretny przykład na HEADLINES przy budżecie $6.50 (GPT-4 kosztuje $33):

```
Kaskada: GPT-J → J1-L → GPT-4
Progi:   akceptuj GPT-J jeśli score > 0.96
         akceptuj J1-L  jeśli score > 0.37
         inaczej                  → GPT-4

Wynik:   80% tańsze + 1.5% lepsze od samego GPT-4
```

Zaskakujący wynik: tańsze modele czasem odpowiadają poprawnie tam gdzie GPT-4 się myli. Kaskada może być **lepsza** niż sam GPT-4, nie tylko tańsza.

---

6. Ograniczenia

- Wymaga labeled data do trenowania Scoring Function
- Cascade działa sekwencyjnie — wolniejsza niż pojedyncze wywołanie
- Przy każdym zapytaniu odpytuje przynajmniej jeden model zanim zdecyduje
- Wymaga rekalibracji gdy zmieniasz modele lub rozkład zapytań

---

7. Różnica między FrugalGPT a RouteLLM

```
RouteLLM:    równoległy — od razu wybiera jeden model
             ocenia tylko ZAPYTANIE (przed wywołaniem)
             szybszy — zero dodatkowych wywołań

FrugalGPT:   sekwencyjny — próbuje modeli po kolei
             ocenia ODPOWIEDŹ (po wywołaniu)
             dokładniejszy — widzi wynik zanim zdecyduje
             wolniejszy — zawsze co najmniej jedno wywołanie
```

---

8. Znaczenie dla pracy inżynierskiej

FrugalGPT i RouteLLM to dwa główne punkty odniesienia w literaturze dotyczącej routingu LLM. Własna praca stoi pomiędzy nimi:

- Od RouteLLM: idea klasyfikacji zapytania **przed** wywołaniem modelu, metryki PGR/APGR/CPT
- Od FrugalGPT: idea Scoring Function jako walidacji odpowiedzi, potwierdzenie że oszczędności 60–98% są realistyczne
- Oba artykuły nie adresują zapytań wieloetapowych (Map-Reduce) — to wyróżnik własnego podejścia